

JAVA LAB PROGRAM 5

Employee – Constructor Overloading

Objective

To develop a Java program that demonstrates **constructor overloading** by creating Employee objects using a default constructor, a 2-parameter constructor, a 4-parameter constructor, and a 5-parameter constructor. The program also demonstrates private data members and a method for displaying employee details.

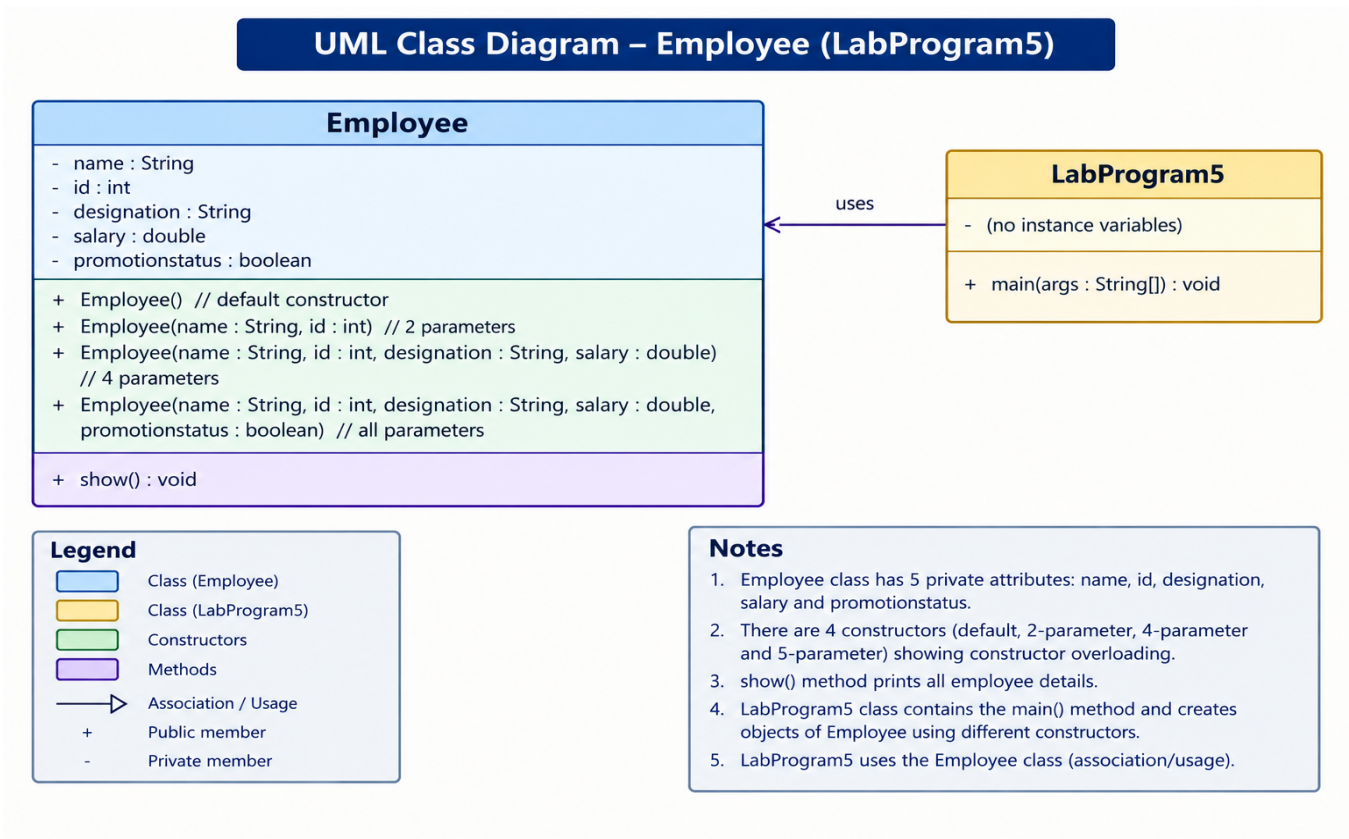
System Requirements

Category	Requirement
Hardware	Computer/Laptop, minimum 4 GB RAM, keyboard and mouse, approximately 1 GB free storage
Operating System	Windows / Linux / macOS
Software	Java Development Kit (JDK 17 or later; JDK 26 is also suitable)
Editor / IDE	VS Code, Eclipse, IntelliJ IDEA, or any Java-compatible editor
Commands	javac for compilation and java for execution

Algorithm

1. Start the program.
2. Define the Employee class with private attributes: name, id, designation, salary, and promotionstatus.
3. Create a default constructor and initialize default employee values.
4. Create a 2-parameter constructor that accepts name and id.
5. Create a 4-parameter constructor that accepts name, id, designation, and salary.
6. Create a 5-parameter constructor that accepts all five employee details.
7. Define the show() method to display employee details.
8. In main(), create four Employee objects using the four overloaded constructors.
9. Call show() for each object.
10. Stop the program.

UML Class Diagram



The UML diagram contains Employee and LabProgram5. Employee has five private attributes, four overloaded constructors, and show(). LabProgram5 contains main() and creates Employee objects. The usage relationship indicates that LabProgram5 uses Employee.

Code Screenshot

VS Code – LabProgram5.java

```
1 | class Employee {
2 |     private String name;
3 |     private int id;
4 |     private String designation;
5 |     private double salary;
6 |     private boolean promotionstatus;
7 |
8 |     public Employee() {
9 |         this.name = "not assigned";
10 |        this.id = 0;
11 |        this.designation = "trainer";
12 |        this.salary = 0.0;
13 |        this.promotionstatus = false;
14 |    }
15 |
16 |    public Employee(String name, int id) {
17 |        this.name = name;
18 |        this.id = id;
19 |        this.designation = "trainer";
20 |        this.salary = 0.0;
21 |        this.promotionstatus = false;
22 |        System.out.println("--- 2 parameters passing constructor");
23 |    }
24 |
25 |    public Employee(String name, int id, String designation, double salary) {
26 |        this.name = name;
27 |        this.id = id;
28 |        this.designation = designation;
29 |        this.salary = salary;
30 |        this.promotionstatus = false;
31 |        System.out.println("4paramters passing constructor");
32 |    }
33 |
34 |    public Employee(String name, int id, String designation,
35 |        double salary, boolean promotionstatus) {
36 |        this.name = name;
37 |        this.id = id;
38 |        this.designation = designation;
39 |        this.salary = salary;
40 |        this.promotionstatus = promotionstatus;
41 |        System.out.println("all details passing constructor");
42 |    }
43 |
44 |    public void show() {
45 |        System.out.println("--- employee details ---");
46 |        System.out.println("name : " + name);
47 |        System.out.println("id : " + id);
48 |        System.out.println("designation : " + designation);
49 |        System.out.println("salary : " + salary);
50 |        System.out.println("promotionstatus : " + promotionstatus);
51 |    }
52 | }
53 |
54 | public class LabProgram5 {
55 |     public static void main(String[] args) {
56 |         Employee e1 = new Employee();
57 |         e1.show();
58 |
59 |         Employee e2 = new Employee("hemanth", 244);
60 |         e2.show();
61 |
62 |         Employee e3 = new Employee("hemanth", 244,
63 |             "software engineer", 70.000);
64 |         e3.show();
65 |
66 |         Employee e4 = new Employee("hemanth", 244,
67 |             "software engineer", 75.000, false);
68 |         e4.show();
69 |     }
70 | }
```

Figure 1: LabProgram5.java source code.

Compilation, Execution and Output

Command Prompt – Compilation & Execution

```
1 | C:\JavaPrograms> javac LabProgram5.java
2 |
3 | C:\JavaPrograms> java LabProgram5
4 | --- employee details ---
5 | name : not assigned
6 | id :0
7 | designation : trainer
8 | salary :0.0
9 | promotionstatus : false
10 | --- 2 parameters passing constructor
11 | --- employee details ---
12 | name : hemanth
13 | id :244
14 | designation : trainer
15 | salary :0.0
16 | promotionstatus : false
17 | 4parameters passing constructor
18 | --- employee details ---
19 | name : hemanth
20 | id :244
21 | designation : software engineer
22 | salary :70.0
23 | promotionstatus : false
24 | all details passing constructor
25 | --- employee details ---
26 | name : hemanth
27 | id :244
28 | designation : software engineer
29 | salary :75.0
30 | promotionstatus : false
31 |
32 | C:\JavaPrograms>
```

Figure 2: Representative terminal screenshot showing javac compilation followed by java execution and the program output.

Result

The Java program successfully demonstrates constructor overloading. Four Employee objects are created using different constructors, and their details are displayed using show().